Dyer
Innovation

CLAUDE CERTIFIED ARCHITECT · SCENARIO 4

# Multi-Agent Research System

A study guide for hub-and-spoke orchestration, subagent context isolation, MCP structured errors, and provenance through synthesis.

LAST UPDATED

April 27, 2026

# Contents


01 Exam Context and This Scenario 3

1.1 Domain Mapping 3
1.2 Scenario Architecture 3

02 Architectural Patterns 4

2.1 2.1 Hub-and-Spoke (Mediator) Pattern 4
2.2 2.2 Subsidiarity in Error Handling 4
2.3 2.3 Structured Error Context 4
2.4 2.4 Semantic Error Classification 4
2.5 2.5 Principle of Least Privilege for Agent Tools 4
2.6 2.6 Tool Description Disambiguation 4
2.7 2.7 Context Management Patterns 4

03 The Agentic Loop Lifecycle (Task 1.1) 7

3.1 How It Works 7
3.2 Example Flow 7
3.3 Anti-Patterns to Know 7

04 Subagent Spawning and Context Passing (Task 1.3) 9

4.1 The Task Tool 9
4.2 Isolated Context — Critical Concept 9
4.3 AgentDefinition Configuration 9
4.4 Parallel Subagent Execution 9
4.5 Structured Context Passing with Metadata 9
4.6 Fork-Based Session Management 9

05 Agent SDK Hooks (Task 1.5) 12

5.1 PostToolUse Hooks — Data Normalization 12

# 01 Exam Context and This Scenario

The Claude Certified Architect – Foundations (CCA-F) is a 60-question, 120-minute proctored exam covering five weighted domains. The **Multi-Agent Research System** is one of six possible exam scenarios, four of which are randomly selected for each candidate. This scenario tests your ability to design a coordinator-subagent architecture where specialized agents (web search, document analysis, synthesis, report generation) collaborate to produce cited research reports.

**This guide focuses on three domains the scenario emphasizes most heavily:**

- **D1 orchestration patterns** — the agentic loop, hub-and-spoke delegation, hooks, session management
- **D2 tool design** — descriptions, `tool_choice`, MCP error responses, server scoping
- **D5 context management** — lost-in-the-middle, provenance, confidence calibration

## Domain Mapping

This scenario primarily tests three domains:

- **Domain 1: Agentic Architecture & Orchestration (27%)**
  - Agentic loop lifecycle
  - Task decomposition
  - Hub-and-spoke coordination
  - Error propagation
  - Hooks
  - Session management
  - Multi-agent pipeline design
- **Domain 2: Tool Design & MCP Integration (18%)**
  - Tool descriptions
  - MCP error patterns
  - Tool distribution
  - `tool_choice` configuration
  - MCP server setup

- **Domain 5: Context Management & Reliability (15%)**
    - Token budgets
    - Lost-in-the-middle
    - Graceful degradation
    - Provenance tracking
    - Human review workflows
    - Confidence calibration

Together these represent **60% of the total exam.**

# Scenario Architecture

The system uses a hub-and-spoke topology with a Coordinator Agent at the center.

**Key architectural facts:**

- Coordinator delegates to four specialized subagents via the Task tool:
    - Web Search Agent
    - Document Analysis Agent
    - Synthesis Agent
    - Report Generation Agent
- All inter-agent communication flows through the coordinator
- Subagents operate with isolated context — they do not inherit the coordinator's conversation history automatically
- The coordinator must explicitly include relevant findings in each subagent's prompt

# 02 Architectural Patterns

These patterns represent the core mental models the CCA expects across the Multi-Agent Research scenario. The focus throughout is on why distractor answers are tempting and where they break down.

## 2.1 Hub-and-Spoke (Mediator) Pattern

**Definition:** A central coordinator manages all communication between specialized subagents. No subagent communicates directly with another.

**Primary value:**

- Centralized observability
- Consistent error handling
- Controlled information flow

> **TIP**
>
> Exam tip:
>
> When a question asks about the "main advantage" of routing through a coordinator, the answer is always about observability and governance — not performance or serialization.

## 2.2 Subsidiarity in Error Handling

**Definition:** Handle errors at the lowest level capable of resolving them. Subagents recover from transient failures locally; permanent failures escalate with full context.

**Key judgment — distinguish:**

- **Transient failures** (e.g., timeout) → retry locally

- **Permanent failures** (e.g., corrupted PDF) → escalate with context

**Rules:**

- Never retry permanent failures
- Never escalate routine transient failures

## 2.3 Structured Error Context

Errors between agents should include:

- Failure type
- What was attempted
- Any partial results
- Potential alternative approaches

> **TRAP**
>
> **Anti-pattern:**
>
> Generic statuses like "search unavailable" hide valuable context from the coordinator.

## 2.4 Semantic Error Classification

Distinguish:

- **Access failures** (e.g., timeout — answer unknown, may need retry)
- **Valid empty results** (query succeeded, found nothing — not an error)

These require fundamentally different coordinator responses.

## 2.5 Principle of Least Privilege for Agent Tools

- Give agents only the tools needed for their role
- Constrain behavior through tool capabilities (what the tool can do) rather than prompt instructions (what the agent is told to do)
- Structural guardrails beat behavioral guidelines

**This is the single most important mental model for the exam: code guarantees; prompts suggest.**

## 2.6 Tool Description Disambiguation

- Tool names and descriptions are the primary mechanism driving LLM tool selection
- When descriptions overlap (e.g., "analyzes content" vs "analyzes documents"), rename and redescribe both tools to create clear semantic boundaries
- Fix at the definition level, not through compensating classifiers or few-shot examples

## 2.7 Context Management Patterns

**Lost-in-the-middle:**

- LLMs attend more strongly to content at the beginning and end of long inputs
- Mitigate with frontloaded key findings summaries and explicit section headers
- Don't just truncate or reorder

**Fix at the source:**

- When combined outputs exceed downstream budgets, redesign upstream agents to return structured, concise data (key facts, citations, relevance scores) instead of verbose content
- Shift left

**Graceful degradation:**

- Proceed with available data while annotating coverage gaps
- Produce value with honest caveats rather than blocking on completeness or hiding incompleteness

**Coordinator planning:**

- When all agents succeed but output is incomplete, the root cause is the coordinator's task decomposition
- Partition research space upfront to prevent overlap

# 03 The Agentic Loop Lifecycle (Task 1.1)

This is the most fundamental concept in the Agent SDK and the exam expects you to know it cold.

## How It Works

The agentic loop is a while loop that sends messages to Claude, inspects the response's `stop_reason` field, and decides whether to continue or terminate.

**When `stop_reason` is `"tool_use"`:**

- Claude wants to call a tool
- Your code executes the requested tool
- Append the tool result to the conversation history
- Send the updated conversation back to Claude for the next iteration

**When `stop_reason` is `"end_turn"`:**

- Claude has decided it's finished
- The loop terminates and returns the final response to the user or coordinator

## Example Flow

Coordinator sends: "Research the impact of AI on healthcare."

- **Turn 1:** Claude returns `stop_reason: "tool_use"` with a Task tool call to spawn the web search subagent. Code executes the Task tool and appends the subagent's findings as a `tool_result`.
- **Turn 2:** Claude sees the findings, returns `stop_reason: "tool_use"` with another Task call to spawn the document analysis subagent. Code executes and appends results.
- **Turn 3:** Claude returns `stop_reason: "tool_use"` to invoke the synthesis subagent with combined findings.
- **Turn 4:** Claude returns `stop_reason: "end_turn"` with the final synthesized report. Loop terminates.

# Anti-Patterns to Know

**Exam trap — distractors will suggest alternatives to `stop_reason`-based loop control:**

- **Parsing natural language signals** (e.g., checking if the assistant says "I'm done") — fragile, non-deterministic
- **Arbitrary iteration caps** as the primary stopping mechanism (e.g., "stop after 5 iterations") — may cut off the agent before it finishes or waste iterations after it's done
- **Checking for assistant text content** as a completion indicator — Claude may include text alongside `tool_use` blocks, so presence of text doesn't mean it's finished

**Correct approach:**

- Continue on `"tool_use"`
- Terminate on `"end_turn"`
- Deterministic and model-driven

# 04 Subagent Spawning and Context Passing (Task 1.3)

## The Task Tool

In the Claude Agent SDK, subagents are spawned using the **Task** tool.

- This is **not optional** — it is the mechanism for creating subagents
- For a coordinator agent to spawn subagents, its `allowedTools` configuration must include `"Task"`
- Without this, the coordinator cannot delegate

## Isolated Context – Critical Concept

**TRAP**

### Subagents do NOT automatically inherit the coordinator's conversation history.

Each subagent starts with a blank slate. The coordinator must explicitly include all relevant context in the subagent's prompt. This means:

— When invoking the synthesis subagent, the coordinator must pass the complete findings from the web search and document analysis agents directly in the prompt

— Subagents do not share memory between invocations. Each Task call creates an independent agent instance

— If the coordinator wants the synthesis agent to know what the web search agent found, it must copy those findings into the synthesis agent's task prompt

## AgentDefinition Configuration

Each subagent type is configured with an `AgentDefinition` that specifies:

- **Description** — used by the coordinator to decide which subagent to invoke
- **System prompt** — defining the subagent's role and behavior
- **Tool restrictions** — limiting which tools the subagent can access

This is where you implement least privilege at the agent level.

# Parallel Subagent Execution

- The coordinator can spawn multiple subagents in parallel by emitting multiple Task tool calls in a single response
- Rather than waiting for the web search agent to finish before spawning the document analysis agent, the coordinator can request both in the same turn
- The SDK executes them concurrently and returns both results in the next iteration

**Exam application:**

- Coordinator should emit Task calls for both web search and document analysis in one turn (they're independent)
- Then emit the synthesis Task only after both complete (it depends on their outputs)
- Common exam pattern: parallel for independent tasks, sequential for dependent ones

# Structured Context Passing with Metadata

When passing findings between agents, use structured data formats that separate content from metadata. Each finding should include:

- The claim/fact
- The source URL or document name
- Page number or section reference
- Publication date

This preserves attribution through the pipeline and enables the synthesis agent to produce properly cited output.

**TRAP**

Exam trap:

— An answer that has the coordinator pass "a summary of findings" without structured metadata will lose source attribution

— The correct answer always preserves claim-source mappings through every handoff

# Fork-Based Session Management

- The `fork_session` API creates independent branches from a shared analysis baseline
- In the research scenario, the coordinator might fork after initial data gathering to explore two different synthesis approaches in parallel, then compare results
- Each fork has its own conversation history from the fork point forward but shares the history before the fork

# 05 Agent SDK Hooks (Task 1.5)

**Hooks are programmatic interceptors that run before or after tool execution. They provide deterministic guarantees that prompt instructions cannot.**

## PostToolUse Hooks – Data Normalization

A `PostToolUse` hook intercepts tool results before the model processes them. In the research system, different MCP tools may return data in different formats:

- One source returns dates as Unix timestamps
- Another as ISO 8601 strings
- A third as "March 2024"

A `PostToolUse` hook normalizes all dates to ISO 8601 before the agent sees them, ensuring consistent downstream processing.

**EXAMPLE**

Example application:

— The web search agent's search tool returns results with numeric relevance scores (0–100)

— The document analysis agent's extraction tool returns relevance as "high/medium/low"

— A `PostToolUse` hook on the document analysis tool converts these to numeric scores, so the synthesis agent receives uniformly formatted inputs

## Tool Call Interception Hooks – Compliance Enforcement

These hooks intercept outgoing tool calls before they execute. They enforce business rules with guaranteed compliance. Examples:

- Block any subagent from making more than 10 API calls per task (cost control)
- Prevent the web search agent from querying domains on a restricted list (compliance)

# When to Choose Hooks Over Prompts

**Use hooks when:**

- The rule requires 100% compliance
- Violations have serious consequences (financial, legal, safety)
- The rule is a simple conditional check (amount > threshold, domain in blocklist)

**Use prompts when:**

- The guidance is nuanced
- Requires judgment
- Involves soft preferences rather than hard rules (e.g., "prefer recent sources over older ones")

**TRAP**

### Exam trap:

— Any answer that relies on prompt instructions for a business rule requiring guaranteed compliance is wrong

— The exam consistently tests whether you choose programmatic enforcement (hooks, prerequisite gates) over prompt-based guidance for critical rules

# 06 Programmatic Prerequisites and Handoffs (Task 1.4)

## Prerequisite Gates

- A programmatic prerequisite blocks a downstream tool call until a prerequisite step has completed
- In the research system, you might require that the coordinator's quality check tool cannot be invoked until both the web search and document analysis subagents have returned results
- This is enforced in code, not in the prompt

**EXAMPLE**

### Classic exam example:

— In the Customer Support scenario (which overlaps conceptually), `process_refund` is blocked until `get_customer` has returned a verified customer ID

— Same pattern applies here: the synthesis Task is blocked until search and analysis Tasks have returned

— **Key insight:** prompt instructions ("always verify before refunding") have a non-zero failure rate; code prerequisites have a zero failure rate

## Structured Handoff Summaries

When a subagent cannot resolve an issue and escalates to the coordinator (or when the system escalates to a human), the handoff must include structured context:

- What was investigated
- What was found
- What failed
- What the recommended next step is

**Contrast:**

- A reviewer who receives "research failed" has nothing to work with
- A reviewer who receives "Web search completed for 3/5 source categories; patent databases timed out after 30s; 15 academic papers retrieved; recommend retrying patent search with narrower date range" can act immediately

# 07 Task Decomposition Strategies (Task 1.6)

## Fixed Pipelines vs Dynamic Decomposition

The exam tests whether you can choose the right decomposition strategy for a given workflow:

**Prompt chaining (fixed sequential pipeline):**

- Use when the workflow is predictable and the steps are known in advance
- Example: for each document, run extraction → validation → enrichment
- Each step's output feeds the next
- Good for structured, repeatable tasks like processing a batch of invoices

**Dynamic adaptive decomposition:**

- Use when the research topic is open-ended and subtasks depend on what's discovered
- Example: the coordinator starts by searching broadly, discovers that AI in music is a major subtopic, then spawns a targeted subagent to investigate music specifically
- The plan adapts based on intermediate findings

**Multi-pass review pattern:**

- For large inputs, split into per-item local analysis passes plus a separate cross-item integration pass
- In the research system: analyze each document individually for key findings, then run a cross-document synthesis pass that identifies themes, conflicts, and gaps across all sources
- This avoids attention dilution

## Iterative Refinement Loops

The coordinator follows this loop:

1. Synthesize
2. Evaluate output for coverage gaps
3. Identify gaps

4. Re-delegate to search and analysis subagents with targeted queries
5. Re-invoke synthesis
6. Repeat until coverage is sufficient

**TIP**

### Exam tip:

Recognize this pattern versus a single-pass pipeline.

# 08 Session State, Resumption, and Forking (Task 1.7)

These are Claude Code-specific features that apply when the research system uses Claude Code as the execution environment.

## Named Session Resumption

- Use `--resume <session-name>` to continue a specific prior conversation
- In a multi-day research task, you might name the session "ai-healthcare-research" and resume it the next day
- The model picks up where it left off with the full conversation history

**When to resume vs start fresh:**

- **Resume** when prior context is mostly valid
- **Start fresh** with an injected summary when prior tool results are stale (e.g., the documents have been updated since the last session, or web search results may have changed)
- **Risk:** stale tool results in a resumed session can lead the model to make decisions based on outdated information

## fork_session

- Creates independent branches from a shared analysis baseline
- After the coordinator gathers initial research, it might fork to explore two synthesis strategies:
  - One that prioritizes recency
  - Another that prioritizes source authority
- Each fork proceeds independently
- The coordinator can then compare the forked outputs and select the better approach

**Exam application:** If asked about exploring divergent approaches from a shared baseline without contaminating one exploration with the other → `fork_session` is the answer.

# 09 MCP Structured Error Responses (Task 2.2)

## The isError Flag

- MCP tool responses include an `isError` boolean flag
- When a tool fails, set `isError: true` and include structured metadata in the response
- The agent uses this flag to determine that the tool call failed and that it needs to reason about recovery

## Structured Error Metadata

A well-designed MCP error response includes three fields:

- `errorCategory` — one of:
  - **transient** (timeout, rate limit — may succeed on retry)
  - **validation** (bad input — fix the input)
  - **permission** (access denied — escalate)
  - **business** (policy violation — explain to user)
- `isRetryable` — Boolean
  - True for transient errors, false for everything else
  - Prevents the agent from wasting attempts retrying non-retryable failures
- **Human-readable description**
  - Explains what went wrong in terms the agent can relay to the user or coordinator
  - For business errors, includes a customer-friendly explanation

**TRAP**

Exam trap:

— A generic "Operation failed" response without error categorization prevents the agent from making appropriate recovery decisions

— The correct answer always returns structured error metadata

# 10 tool_choice Configuration (Task 2.3)

The `tool_choice` parameter controls how Claude selects tools. This is a high-frequency exam topic.

| SETTING | BEHAVIOR | WHEN TO USE |
|---|---|---|
| `"auto"` | Model may return text instead of calling a tool. It decides whether a tool call is needed. | Default for conversational agents. Use when the model should decide whether to use tools. |
| `"any"` | Model MUST call a tool but can choose which one. Cannot return text-only response. | Guaranteeing structured output when you have multiple extraction schemas and the doc type is unknown. |
| `{"type":"tool","name":"..."}` | Model MUST call the specified tool. No choice involved. | Forcing a specific extraction step first (e.g., `extract_metadata` before enrichment tools). |

## Tool Count and Selection Reliability

- Giving an agent access to too many tools (e.g., 18 instead of 4–5) degrades tool selection reliability by increasing decision complexity
- Each subagent should have only the tools relevant to its role
- If the synthesis agent has access to web search tools alongside its synthesis tools, it will drift into performing searches instead of synthesizing
- **Rule:** restrict tool sets per agent role

# 11 MCP Server Configuration (Task 2.4)

## Scoping: Project vs User Level

**Project-level:**

- Configure in `.mcp.json` at the project root
- Shared with all team members via version control
- Use for production MCP servers that the entire team needs (e.g., the research system's web search server, document store server)

**User-level:**

- Configure in `~/.claude.json`
- Personal to one developer, not shared via version control
- Use for experimental MCP servers or personal tools (e.g., a personal notes server)

## Environment Variable Expansion

- MCP server configurations in `.mcp.json` support environment variable expansion using `${VARIABLE_NAME}` syntax
- Allows credential management without committing secrets to version control
- Example: a search API key would be configured as `${SEARCH_API_KEY}` in `.mcp.json`, with each developer setting the actual key in their local environment

## MCP Resources vs Tools

**MCP resources:**

- Expose content catalogs (e.g., available document summaries, database schemas, issue hierarchies) that the agent can browse without making action-oriented tool calls
- Reduces exploratory tool calls — instead of the agent calling `list_documents` repeatedly to discover what's available, it reads the resource catalog once

**TRAP**

### Exam trap:

— If asked how to reduce the number of exploratory tool calls an agent makes to discover available data → the answer is **MCP resources** (content catalogs)

— Not bigger tool descriptions or more elaborate prompting

# 12 Verbose Tool Output Management (Task 5.1)

## The Problem

- MCP tools often return far more data than the agent needs
- Example: a document lookup tool might return 40+ fields (creation date, last modified, file size, permissions, full revision history) when only 5 are relevant (title, key findings, publication date, author, URL)
- Over multiple tool calls, these verbose responses accumulate in the conversation history and consume token budget disproportionate to their value

## The Fix

- Trim tool outputs to only relevant fields before they enter the conversation context
- This can be done via:
  - `PostToolUse` hooks (best)
  - Designing MCP tools to accept a `fields` parameter that controls which data is returned
- **Goal:** keep context lean and focused

## Progressive Summarization Risks

- When summarizing long conversations to free up context, critical numerical values, percentages, dates, and specific quotes can be lost
- A summary that says "the report showed strong growth" loses the fact that the original said "47.3% year-over-year growth in Q3 2025"
- **Fix:** extract transactional facts into a persistent structured block that survives summarization

# 13 Context Management in Extended Sessions (Task 5.4)

## Context Degradation

- In extended sessions, models start giving inconsistent answers and referencing "typical patterns" rather than specific classes or findings discovered earlier in the conversation
- This happens because earlier tool results get pushed further back in the context window and receive weaker attention

## Scratchpad Files

- Have agents maintain scratchpad files (e.g., `research_findings.md`) that record key findings as they're discovered
- When the agent needs to reference earlier findings, it reads the scratchpad file rather than relying on its degraded memory of earlier conversation turns
- This provides a persistent, reliable store of facts across context boundaries

## Crash Recovery with Manifests

For long-running research pipelines, design structured state persistence:

- Each subagent exports its current state (findings, progress, errors) to a known file location
- The coordinator maintains a manifest file listing all agent states
- If the system crashes mid-research:
    1. The coordinator loads the manifest on resume
    2. Reads each agent's exported state
    3. Injects the relevant summaries into agent prompts to continue where it left off

## /compact Command

- Use `/compact` to reduce context usage during extended exploration sessions when the context fills with verbose discovery output

- This compresses the conversation history while preserving the essential information
- Know that this exists as a tool for managing context in long sessions

# 14 Human Review Workflows and Confidence (Task 5.5)

This task statement applies when the research system needs human oversight for quality assurance.

## Aggregate Metrics Can Be Misleading

- A system reporting 97% overall extraction accuracy may be 99.5% accurate on news articles but only 82% accurate on academic papers
- Aggregate accuracy masks poor performance on specific document types or fields
- **Rule:** always validate accuracy by document type and field segment before automating high-confidence extractions

## Stratified Random Sampling

- Even for high-confidence extractions, implement ongoing stratified random sampling
- Randomly select a representative sample across document types and source categories for human review
- This detects novel error patterns that wouldn't be caught by only reviewing low-confidence items

## Field-Level Confidence Scores

- Have the model output confidence scores at the field level, not just the document level
- A research finding might be high-confidence for the claim but low-confidence for the publication date
- Calibration workflow:
    1. Calibrate confidence scores using labeled validation sets
    2. Use the calibrated thresholds to route extractions:
        - High-confidence items → proceed automatically
        - Low-confidence items → human review

**TRAP**

### Exam trap:

— LLM self-reported confidence scores are poorly calibrated out of the box

— The correct answer always involves calibrating against labeled data

— Never trust raw confidence scores directly

# 15 Information Provenance (Task 5.6)

## Claim-Source Mappings

- Source attribution is lost during summarization when findings are compressed without preserving who said what
- **Fix:** require subagents to output structured claim-source mappings where each finding includes:
    - The claim text
    - The source (URL, document, page)
    - A relevant excerpt
- The synthesis agent must preserve and merge these mappings — never discard them during combination

## Temporal Data Handling

- Require subagents to include publication or data collection dates in their structured outputs
- Without dates, the synthesis agent may misinterpret temporal differences as contradictions
- **Example:** if Source A says "12% growth" (2023 data) and Source B says "40% growth" (2025 data), these aren't conflicting — they're measuring different time periods
- Dates enable correct interpretation

## Content-Type-Appropriate Rendering

Different content types should be rendered appropriately in synthesis outputs:

- **Financial data** → tables
- **News findings** → prose narrative
- **Technical findings** → structured lists

**TRAP**

### Anti-pattern:

Converting everything to a uniform format (e.g., all bullet points) loses the natural structure that makes each content type readable.

# 16 Anti-Patterns to Recognize on Sight

The eleven failure modes below are the patterns the exam reaches for when constructing distractor answers. Each describes the surface symptom, the underlying mistake, and why the correct architectural answer rejects it.

| ANTI-PATTERN | WHAT IT LOOKS LIKE | WHY IT'S WRONG |
|---|---|---|
| **Silent failure** | Return empty result as successful; include conflicts without flagging them | Corrupts downstream analysis; pipeline looks healthy while output is wrong |
| **Crash-the-plane** | Any error terminates entire workflow | Disproportionate; coordinator exists for nuanced recovery |
| **Retry permanent failures** | Exponential backoff on corrupted files | Corruption won't self-heal; wastes time and resources |
| **Downstream compensation** | Adding summarization agents, vector DBs, or deduplication for upstream problems | Adds complexity; fix at source instead (shift left) |
| **Behavioral over structural** | Prompt says "only use for documents" instead of scoping the tool | LLMs don't consistently follow constraints; structural guardrails provide guarantees |
| **Over-engineering** | Dedicated error-handling agents, shared state systems, pre-routing classifiers | Each component is a new failure mode; simplest correct solution wins |
| **NL loop termination** | Parsing "I'm done" text instead of checking `stop_reason` | Fragile, non-deterministic; `stop_reason` is the authoritative signal |
| **Too many tools** | Agent has 18 tools instead of 4–5 | Degrades selection reliability; increases misrouting |
| **Generic error status** | "Operation failed" without `errorCategory` or `isRetryable` | Coordinator can't make informed recovery decisions |
| **Raw confidence trust** | Using model's self-reported confidence without calibration | LLM confidence is poorly calibrated; calibrate against labeled validation data |

# 17 Core Terminology

The vocabulary below appears throughout the exam — knowing each term cold lets you parse the question stem quickly and reject answer choices that misuse the API surface.

| TERM | DEFINITION |
| --- | --- |
| `stop_reason` | API response field indicating why Claude stopped generating. `"tool_use"` = wants to call a tool (loop continues). `"end_turn"` = finished (loop terminates). |
| **Task tool** | The Agent SDK mechanism for spawning subagents. Coordinator's `allowedTools` must include `"Task"` to delegate. |
| `allowedTools` | Configuration restricting which tools an agent can use. Enforces least privilege at the agent level. |
| `AgentDefinition` | SDK configuration for a subagent: description, system prompt, and tool restrictions. |
| `PostToolUse` **hook** | Programmatic interceptor that transforms tool results before the model processes them. Used for data normalization. |
| `fork_session` | Creates independent conversation branches from a shared baseline to explore divergent approaches. |
| `isError` / `isRetryable` | MCP error response fields. `isError` flags failure; `isRetryable` indicates if retry may succeed. |
| `errorCategory` | MCP field classifying errors as transient, validation, permission, or business. |
| `tool_choice` | API parameter: `"auto"` (may skip tools), `"any"` (must call a tool), or forced (must call specific tool). |
| `.mcp.json` | Project-level MCP server configuration. Shared via version control. Supports `${ENV_VAR}` expansion. |
| **MCP resources** | Content catalogs exposed by MCP servers. Agents browse them to discover available data without exploratory tool calls. |
| **Hub-and-spoke** | Topology where all agent communication routes through a central coordinator for observability and governance. |
| **Claim-source mapping** | |

# 18 15 Guiding Principles

When uncertain on an exam question, run it through these. The correct answer will align with most.

1. **Code guarantees; prompts suggest.** Enforce via hooks, prerequisites, and tool capabilities — not instructions.
2. `stop_reason` **is the loop signal.** Continue on `"tool_use"`, terminate on `"end_turn"`. Nothing else.
3. **Fix problems at the source.** Redesign upstream outputs rather than adding downstream compensation.
4. **Errors are actionable data.** Include failure type, what was attempted, partial results, and alternatives.
5. **Handle errors at the right level.** Transient locally; permanent escalated with context.
6. **Classify failures semantically.** Access failure ≠ valid empty result ≠ data conflict ≠ partial success.
7. **Context must be explicit.** Subagents don't inherit history. Pass findings directly in prompts.
8. **The coordinator plans; agents execute.** Incomplete output with successful agents = bad decomposition.
9. **Produce value with transparency.** Work with what's available, annotate what's missing.
10. **Least privilege for tools.** 4–5 scoped tools per agent. Too many degrades selection.
11. **Simplest correct solution wins.** Avoid added agents, shared state, classifiers when simpler fixes work.
12. **The hub exists for observability.** Coordinator's value is visibility and governance, not performance.
13. **Structured error responses always.** `isError` + `errorCategory` + `isRetryable` + description.
14. **Calibrate before trusting confidence.** Raw LLM confidence is unreliable; validate against labeled data.
15. **Preserve provenance through every handoff.** Claim-source mappings, dates, and excerpts must survive synthesis.

End of Study Guide

You've written a function. Now we're going to make it fail on purpose, and then fix it. This is how you learn to debug.